Insure Well

Health Insurance Management System

Software Requirements Specification (SRS)

Version 1.0
March 2026
Prepared for: Capstone Project Review
Technology Stack: Angular 21 + Spring Boot 3.4.5 (Java 21)

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Project Overview 1
1.3 Scope 2
1.4 Document Conventions 2
1.5 Intended Audience 3
1.6 Project References 3
2. Overall System Description 4
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes and Characteristics 5
3. System Architecture 6
3.1 Technology Stack 6
3.2 Backend Layered Architecture 7
3.3 Frontend Route Architecture 7
3.4 User Entity Model 8
4. Functional Requirements 9
4.1 Authentication and Authorization (FR-AUTH) 9
4.2 Insurance Plan Management (FR-PLAN) 10
4.3 Premium Calculation (FR-PREM) 11
4.4 Policy Lifecycle Management (FR-POL) 12
4.5 Underwriter Workflow (FR-UW) 13
4.6 Payment Processing (FR-PAY) 14
4.7 Claims Management (FR-CLM) 15
4.8 Admin Management (FR-ADM) 16
4.9 Real-Time Notifications (FR-NOTIF) 17
4.10 PDF Generation (FR-PDF) 17
4.11 Audit Logging (FR-AUDIT) 18
5. Non-Functional Requirements 18
5.1 Security (NFR-SEC) 18
5.2 Performance (NFR-PERF) 19
5.3 Reliability (NFR-REL) 19
5.4 Scalability (NFR-SCALE) 20
5.5 Usability (NFR-USE) 20
5.6 Maintainability (NFR-MAIN) 20
5.7 Portability (NFR-PORT) 21

6. Complete API Specification 21
6.1 Authentication 21
6.2 Insurance Plans 22
6.3 Premium Calculation 22
6.4 Policies 23
6.5 Admin Operations 24
6.6 Underwriter Operations 25
6.7 Claims 26
6.8 Claims Officer Operations 27
6.9 Payments and Razorpay 28
6.10 Notifications, Audit, and PDF 28
7. Environment and Configuration 29
7.1 Development Environment 29
7.2 Production Environment Variables 29
7.3 Security Notice 30
7.4 Local Setup Guide 30
8. Data Storage and File Management 31
8.1 Database Entity Model 31
8.2 File Storage Strategy 32
9. Constraints and Assumptions 33
9.1 Constraints 33
9.2 Assumptions 33
10. Document Revision History 34

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Insure Well Health Insurance Management System. It serves as the authoritative reference for developers, testers, architects, and stakeholders throughout the project lifecycle.

The document captures the system's intended behavior, constraints, user roles, business logic, data design, and interface specifications, ensuring all parties share a single, unambiguous understanding of what the system must do and how it must perform.

1.2 Project Overview

Insure Well is a full-stack, enterprise-grade digital platform designed to modernize and automate the entire health insurance lifecycle. Historically, the insurance sector has relied on manual data entry, physical document handling, and fragmented workflows, which led to delayed claim settlements, opaque status tracking, and high administrative costs.

Insure Well replaces this with a centralized, role-driven digital infrastructure covering customer onboarding, insurance plan management, dynamic premium calculation, underwriter risk review, multi-stage claim processing, and payment management, all delivered through a secure RESTful API consumed by a modern Angular frontend.

1.3 Scope

The system covers the following functional domains:

- User registration, authentication, and role-based access control
- Insurance plan browsing and management
- Premium quote calculation considering members, age, and pre-existing conditions
- Policy purchasing, underwriter review, and lifecycle tracking
- Payment processing with standard and Razorpay gateway integration
- Multi-stage claim submission, officer review, and financial settlement
- Document upload for health check reports and claims evidence
- Real-time notifications via Server-Sent Events (SSE)
- Audit logging of all significant system actions
- PDF generation for policy certificates and payment receipts

Out of scope for version 1.0: AI-based medical risk prediction, third-party hospital system integration, and mobile native applications.

1.4 Document Conventions

Term	Description
SRS	Software Requirements Specification
JWT	JSON Web Token — used for stateless authentication
RBAC	Role-Based Access Control
SSE	Server-Sent Events — used for real-time push notifications
OTP	One-Time Password — used in the forgot-password flow
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
UW	Underwriter
H2	In-memory database used for local development
MySQL	Relational database used in production

1.5 Intended Audience

- Software Developers — Frontend (Angular 21) and Backend (Spring Boot 3.4.5)
- System Architects — Reviewing system design, integration, and security boundaries
- Quality Assurance Engineers — Deriving test cases from requirements
- Project Managers and Stakeholders — Tracking scope and delivery progress
- Academic Evaluators — Capstone project review and grading

1.6 Project References

- Repository: Capstone/ (Angular Frontend + Spring Boot Backend)
- Frontend directory: Frontend- Insure Well / (Angular 21 SPA)
- Backend directory: Backend- Insure Well / (Spring Boot 3.4.5, Java 21)
- API Base URL (Development): http://localhost:8080/api
- Swagger UI (Development): http://localhost:8080/swagger-ui.html
- H2 Console (Development): http://localhost:8080/h2-console
- Frontend UI (Development): http://localhost:4200

2. Overall System Description

2.1 Product Perspective

Insure Well is a standalone, web-based application composed of two primary subsystems: (1) a Frontend Single-Page Application built with Angular 21, served at http://localhost:4200, and (2) a Backend REST API built with Spring Boot 3.4.5, served at http://localhost:8080/api. The two subsystems communicate exclusively via RESTful HTTP APIs using JSON. Authentication is managed using JWT access and refresh tokens stored in the browser's localStorage. Real-time notifications are delivered over Server-Sent Events (SSE).

2.2 Product Functions

The following table summarizes the high-level functions of the Insure Well system:

#	Function	Description
1	**User Authentication**	JWT-based login/register, refresh tokens, OTP-based password reset
2	**Plan Management**	Admin creates and manages insurance plans; customers browse publicly
3	**Premium Calculation**	Quote engine considering member count, age, and pre-existing conditions
4	**Policy Purchase**	Customer applies for a policy with optional health check document
5	**Underwriter Workflow**	Assigned underwriter reviews risk, generates premium quote, raises concerns
6	**Payment Processing**	Standard payment endpoint and Razorpay payment gateway integration
7	**Claims Management**	Customer files claims; officer reviews, makes decisions, and settles
8	**Admin Dashboard**	User activation, plan management, and workflow assignment controls
9	**Notifications (SSE)**	Real-time server-sent push notifications with read/unread state
10	**Audit Logging**	Tracks all significant role-based system actions for compliance
11	**PDF Generation**	Downloadable policy certificates and payment receipts using iText

2.3 User Classes and Characteristics

The system implements four distinct user roles managed via Role-Based Access Control (RBAC). Role claims are embedded in JWT tokens. The backend Spring Security layer is the authoritative security boundary; the Angular route guards serve as the UX-level access layer only.

2.3.1 Customer

End users who purchase and manage health insurance policies. Customers interact with the system for plan browsing, policy application, premium payment, and claims filing.

- Self-registration and JWT-based login
- Browse active insurance plans (public, no login required)
- Calculate premium quotes as a guest or as an authenticated user
- Purchase policies with optional health check document upload
- Track policy status through the complete lifecycle
- Make payments via the standard endpoint or Razorpay gateway
- File claims with supporting medical documents
- Track claim status and view settlement details
- Manage personal profile and view notification history

2.3.2 Admin

System administrators who manage users, insurance plans, and workflow assignments. Admins have the broadest system access.

- View system-wide dashboard with key metrics
- Create and manage underwriter and claims officer accounts
- Activate or deactivate customer accounts
- Create, update, activate, and deactivate insurance plans
- Assign underwriters to pending policy applications
- Assign claims officers to submitted claims
- View all payments, policies, and claims system-wide

2.3.3 Underwriter

Risk assessment specialists who review customer policy applications and generate premium quotes.

- View all policies assigned to them, categorized by status
- Calculate risk and generate a detailed premium quote with factor breakdown
- Send the premium quote to the customer, transitioning policy to QUOTE_SENT
- Raise concerns on an application, requiring the customer to reapply
- View personal dashboard, profile, and available insurance plans

2.3.4 Claims Officer

Specialists responsible for reviewing, deciding on, and settling insurance claims.

- View the queue of submitted and unassigned claims
- Self-assign (pick up) claims from the queue for review
- Review claim details and supporting medical documents
- Make claim decisions: Approve, Partially Approve, Reject, Document Pending, or Escalate
- Initiate claim settlement and track settlement progress

3. System Architecture

3.1 Technology Stack

3.1.1 Frontend

Technology	Version	Purpose
Angular	^21.2.x	SPA framework with standalone router and component config
RxJS	~7.8	Reactive programming and HTTP client observable streams
TypeScript	Latest	Statically typed primary frontend language
JWT (localStorage)	—	Auth token storage using keys: hs_token, hs_refresh, hs_user
HTTP Interceptor	—	Automatically attaches Bearer JWT to all outgoing API requests
EventSource (SSE)	—	Real-time server-sent notification subscription

3.1.2 Backend

Technology	Version	Purpose
Spring Boot	3.4.5	Core REST API framework
Spring Security	—	RBAC enforcement, JWT filter, CORS configuration
Spring Data JPA/Hibernate	—	ORM layer and database access via repositories
Java	21	Primary backend language (LTS)
JJWT (io.jsonwebtoken)	—	JWT creation, signing, and verification
SpringDoc OpenAPI	—	Auto-generated Swagger UI at /swagger-ui.html
Spring Mail	—	OTP emails and system notification delivery
iText	5.5.13.3	PDF generation for policy certificates and receipts
Razorpay Java SDK	1.4.5	Payment gateway: order creation and HMAC verification
H2 (development)	—	In-memory database, auto-recreated each run
MySQL (production)	8.x	Relational database for production environment

3.2 Backend Layered Architecture

The backend follows a standard layered architecture. Each layer has a single, well-defined responsibility:

Layer	Package	Responsibility
Controllers	com.insurewell.controller	REST endpoints, RBAC annotations, HTTP request/response handling
Services	com.insurewell.service	Core business logic, state machine transitions, data validation
Repositories	com.insurewell.repository	JPA interfaces for CRUD database operations and custom queries
Entities	com.insurewell.entity	JPA/Hibernate persistence model mapping to database tables
DTOs	com.insurewell.dto.*	Request and response contracts, decoupled from database entities
Security/Config	com.insurewell.config, .util	JWT utilities, security filters, CORS config, OpenAPI setup

The standard request flow is: HTTP Request → Controller → Service → Repository → Database. DTOs are used at the controller boundary; entities are used internally within the service and repository layers.

3.3 Frontend Route Architecture

Routes are defined in app.routes.ts with route guards applied per role. Public routes require no authentication. Role-specific routes require a valid JWT with the matching role claim.

Route Prefix	Required Role	Key Routes
/	Public (no auth)	/, /login, /register, /plans, /forgot-password
/admin/*	ADMIN	/dashboard, /customers, /underwriters, /claims-officers, /plans, /policies, /assign-underwriter, /claims, /assign-claims-officer, /profile
/customer/*	CUSTOMER	/dashboard, /plans, /calculator, /policies, /claims, /claims/new, /payment/:policyId, /profile
/underwriter/*	UNDERWRITER	/dashboard, /plans, /pending, /send-quote/:policyId, /policies, /profile
/claims-officer/*	CLAIMS_OFFICER	/dashboard, /my-claims, /review/:claimId, /profile

3.4 User Entity Model

The user model uses single-table inheritance in JPA. A base User entity holds common fields, and four subclasses represent each role:

- User (base): id, email, password (BCrypt hashed), role, isActive, createdAt
- Customer extends User: demographics, address, phone, linked policies and claims
- Admin extends User: administrative access, no additional domain fields
- Underwriter extends User: employeeId, department, assigned policies
- ClaimsOfficer extends User: employeeId, department, assigned claims

4. Functional Requirements

4.1 Authentication and Authorization (FR-AUTH)

FR-AUTH-01: User Registration

The system shall allow new customers to self-register by providing their full name, email address, date of birth, phone number, and password. The backend shall hash the password using BCrypt and issue a JWT access token and refresh token upon successful registration.

Endpoint	POST /api/auth/register
Actor	Customer (unauthenticated)
Input	name, email, password, dateOfBirth, phone
Output	JWT access token, refresh token, user profile (id, role, name)
Precondition	Email address must not already be registered in the system
Postcondition	Customer account created with ACTIVE status; JWT tokens issued

FR-AUTH-02: User Login

The system shall authenticate registered users via email and password. On success, a JWT access token and refresh token shall be returned. The role claim embedded in the JWT shall be used by the frontend to navigate the user to their role-specific dashboard.

Endpoint	POST /api/auth/login
Actor	Customer, Admin, Underwriter, Claims Officer
Input	email, password
Output	JWT access token, refresh token, user info (id, role, name, email)
Error Cases	HTTP 401 if credentials are invalid; HTTP 403 if account is deactivated

FR-AUTH-03: Token Refresh

The system shall provide a token refresh endpoint. When the JWT access token (hs_token) expires, the Angular HTTP interceptor shall automatically call this endpoint using the stored refresh token (hs_refresh) to obtain a new access token without requiring the user to log in again. Returns HTTP 401 if the refresh token is expired or invalid.

- Endpoint: POST /api/auth/refresh

FR-AUTH-04: Forgot Password via Email OTP

The system shall support a two-step password recovery flow. In step one, the user provides their registered email and the system sends a time-limited OTP via email. In step two, the user submits the OTP along with a new password to complete the reset.

- Step 1 — POST /api/auth/forgot-password: Accepts email; triggers OTP delivery via SMTP
- Step 2 — POST /api/auth/reset-password: Accepts email, otp, and newPassword; updates credentials

FR-AUTH-05: Role-Based Access Control

All protected API endpoints shall enforce RBAC using Spring Security annotations on controller methods. The JWT role claim embedded at login time determines which resources and operations are accessible to each user. The backend is the authoritative security boundary. Angular route guards serve as a UX-layer complement only and do not replace backend security.

4.2 Insurance Plan Management (FR-PLAN)

FR-PLAN-01: Public Plan Browsing

Any user, including unauthenticated guests, shall be able to browse all active insurance plans and view individual plan details without logging in.

- GET /api/plans — Returns a list of all active insurance plans
- GET /api/plans/{id} — Returns full details of a specific plan by its ID
- GET /api/plans/type/{type} — Returns plans filtered by insurance type

FR-PLAN-02: Admin Plan Management

Administrators shall be able to create new insurance plans and manage the lifecycle of existing plans.

- POST /api/admin/plans — Create a new insurance plan
- PUT /api/admin/plans/{id} — Update an existing plan's details
- PATCH /api/admin/plans/{id}/activate — Set plan status to active
- PATCH /api/admin/plans/{id}/deactivate — Set plan status to inactive

4.3 Premium Calculation (FR-PREM)

FR-PREM-01: Guest Premium Calculation

Unauthenticated users shall calculate a premium estimate by providing a plan ID, member details (name, age, relationship), and pre-existing condition flags. The calculation considers member count, age factors, and disease factors.

- Endpoint: POST /api/premium/calculate

FR-PREM-02: Authenticated Premium Calculation

Authenticated customers shall calculate premium quotes that are stored and associated with their account for future reference.

- POST /api/premium/calculate/me — Calculates and stores the quote against the logged-in customer
- GET /api/premium/my-quotes — Retrieves the customer's full quote history

FR-PREM-03: Underwriter Detailed Quote

Underwriters shall access a detailed premium breakdown for assigned policies to support risk-based pricing decisions. The breakdown includes age factor, disease factor, and member count contributions.

- GET /api/underwriter/policy/{policyId}/calculate-quote — Base calculation for a policy
- GET /api/underwriter/calculate-quote/{policyId} — Full breakdown with all contributing factors

4.4 Policy Lifecycle Management (FR-POL)

FR-POL-01: Policy Purchase

Authenticated customers with a valid quote shall be able to apply for a health insurance policy. The application may optionally include a health check report document attached as a multipart form upload.

- POST /api/policies — Standard policy application with JSON request body
- POST /api/policies/with-document — Multipart request containing policy JSON and an optional healthCheckReport file

FR-POL-02: Policy Status Tracking

Customers, admins, and underwriters shall be able to view policies and their current status at any point in the lifecycle.

- GET /api/policies/my-policies — Returns all policies belonging to the authenticated customer
- GET /api/policies — Returns all policies (accessible by Admin and Underwriter roles)
- GET /api/policies/{id} — Returns full details of a specific policy
- GET /api/policies/{id}/members — Returns the list of members covered under a specific policy

FR-POL-03: Policy State Machine

The system shall enforce the following policy status transitions. Transitions are triggered by specific actor actions and are irreversible except where explicitly defined:

From Status	To Status	Triggering Action
PENDING	ASSIGNED	Admin assigns an underwriter to the policy application
ASSIGNED	QUOTE_SENT	Underwriter sends the premium quote to the customer
QUOTE_SENT	ACTIVE	Customer successfully completes premium payment
ASSIGNED / QUOTE_SENT	CONCERN_RAISED	Underwriter raises a concern on the application
CONCERN_RAISED	PENDING	Customer reapplies with corrected information
ACTIVE	CANCELLED	Customer or Admin explicitly cancels the policy
ACTIVE	LAPSED	Premium payment is missed beyond the grace period
ACTIVE	RENEWAL_DUE	Policy term is approaching its expiry date
RENEWAL_DUE	RENEWED	Customer successfully renews the policy
ACTIVE / RENEWED	EXPIRED	Policy term ends without a renewal being processed

FR-POL-04: Policy Actions

- PATCH /api/policies/{id}/cancel — Customer or Admin cancels an active policy
- POST /api/policies/{id}/renew — Customer initiates renewal of a policy in RENEWAL_DUE status
- PUT /api/policies/{id}/reapply — Customer reapplies after a concern is raised (multipart)
- GET /api/policies/{id}/document/download — Downloads the uploaded health check report

4.5 Underwriter Workflow (FR-UW)

FR-UW-01: View Assignments

- GET /api/underwriter/pending-assignments — Returns all policies assigned and awaiting review
- GET /api/underwriter/policies — Returns all policies ever assigned to this underwriter
- GET /api/underwriter/my-policies — Personal aggregated policy view with status filters
- GET /api/underwriter/plans — View available insurance plans for reference

FR-UW-02: Send Premium Quote

After completing the risk assessment, the underwriter shall submit a premium quote. The system shall transition the policy status from ASSIGNED to QUOTE_SENT and notify the customer.

- Endpoint: POST /api/underwriter/policy/{policyId}/send-quote
- Request body includes the final quote amount and optional breakdown commentary

FR-UW-03: Raise Concern

If a customer's documentation or health information is insufficient, the underwriter shall raise a concern. The system transitions the policy to CONCERN_RAISED and notifies the customer to reapply with corrected information.

- Endpoint: POST /api/underwriter/policy/{policyId}/raise-concern
- Request body includes a human-readable reason for the concern

4.6 Payment Processing (FR-PAY)

FR-PAY-01: Standard Payment Endpoint

The system shall provide a direct payment API for processing premium payments without a third-party gateway.

- POST /api/payments — Submit a payment record for a policy; triggers policy activation on success
- GET /api/payments/my-payments — Returns the authenticated customer's full payment history
- GET /api/payments/{id} — Returns details of a specific payment record
- GET /api/payments/policy/{id} — Returns all payment records associated with a specific policy
- GET /api/payments — Returns all payments system-wide (Admin access only)
- GET /api/payments/{id}/receipt — Downloads the PDF payment receipt for a specific payment

FR-PAY-02: Razorpay Integration

The system shall integrate with the Razorpay payment gateway for secure online transactions. Signature verification shall be performed server-side to prevent payment tampering.

- POST /api/razorpay/create-order — Creates a Razorpay order; returns order ID, currency, and amount
- POST /api/razorpay/verify — Verifies the HMAC-SHA256 payment signature; activates policy on success
- POST /api/razorpay/failure — Logs the payment failure event; policy remains in QUOTE_SENT status

FR-PAY-03: Policy Activation on Payment

On successful payment via either channel, the system shall automatically transition the associated policy status to ACTIVE, create an audit log entry, and dispatch a real-time notification to the customer.

4.7 Claims Management (FR-CLM)

FR-CLM-01: Claim Filing

Customers holding an ACTIVE policy shall be able to file insurance claims with optional medical document attachments. Documents are stored on the server filesystem and tracked via the ClaimDocument database entity.

Endpoint	POST /api/claims (multipart/form-data)
Actor	Customer
Input	claimData (JSON part) + optional medical documents (file parts)
Supported Documents	Medical bills, prescriptions, diagnostic lab reports
Storage	Filesystem under uploads/claims/<claimId>/; metadata stored in ClaimDocument entity
Precondition	Customer must have an ACTIVE policy to file a claim

FR-CLM-02: Claim State Machine

The system shall enforce the following claim status transitions:

From Status	To Status	Triggering Action / Actor
SUBMITTED	UNDER_REVIEW	Claims officer picks up and self-assigns the claim
UNDER_REVIEW	APPROVED	Claims officer approves the claim in full
UNDER_REVIEW	PARTIALLY_APPROVED	Claims officer approves a portion of the claimed amount
UNDER_REVIEW	REJECTED	Claims officer rejects the claim with reason
UNDER_REVIEW	DOCUMENT_PENDING	Officer requests additional supporting documents
UNDER_REVIEW	APPEAL_REQUESTED	Customer submits a formal appeal on a decision
APPROVED / PARTIALLY_APPROVED	SETTLEMENT_IN_PROGRESS	Officer or Admin initiates the settlement process
SETTLEMENT_IN_PROGRESS	SETTLED	Settlement is completed and funds are disbursed

FR-CLM-03: Claims Officer Actions

- GET /api/claims-officer/queue — Returns the full queue of submitted, unassigned claims
- POST /api/claims-officer/claim/{id}/pickup — Officer self-assigns a claim; status changes to UNDER_REVIEW
- POST /api/claims-officer/claim/{id}/review — Officer submits a review decision with verdict and reason
- GET /api/claims-officer/my-claims — Returns all claims currently assigned to this officer
- GET /api/claims-officer/my-claims/status/{status} — Returns officer's claims filtered by status
- GET /api/claims-officer/my-decisions — Returns the officer's full historical decision log
- GET /api/claims-officer/claim/{id} — Returns complete details of a specific claim

FR-CLM-04: Claim Document Management

- GET /api/claims/{id}/documents — Lists all documents attached to a specific claim
- GET /api/claims/{claimId}/documents/{docId}/download — Downloads a document by its unique ID
- GET /api/claims/{claimId}/documents/view/{fileName} — Streams a document by filename for inline viewing
- GET /api/claims/{claimId}/documents/download/{fileName} — Downloads a document by filename

FR-CLM-05: Claim Settlement

Settlement may be initiated by an Admin or a Claims Officer once a claim has reached APPROVED or PARTIALLY_APPROVED status.

- Endpoint: POST /api/claims/{claimId}/settle
- Status transitions on settlement: APPROVED → SETTLEMENT_IN_PROGRESS → SETTLED

4.8 Admin Management (FR-ADM)

FR-ADM-01: User Account Management

- POST /api/admin/create-underwriter — Creates a new underwriter account
- POST /api/admin/create-claims-officer — Creates a new claims officer account
- GET /api/admin/customers — Returns a list of all customer accounts
- GET /api/admin/underwriters — Returns a list of all underwriter accounts
- GET /api/admin/claims-officers — Returns a list of all claims officer accounts
- PATCH /api/admin/users/{id}/activate — Activates a previously deactivated account
- PATCH /api/admin/users/{id}/deactivate — Deactivates an account, blocking future logins

FR-ADM-02: Workflow Assignment

- GET /api/admin/pending-applications — Returns policies in PENDING status awaiting underwriter assignment
- POST /api/admin/policies/{policyId}/assign-underwriter — Assigns an underwriter; policy transitions to ASSIGNED

- GET /api/admin/submitted-claims — Returns claims in SUBMITTED status awaiting officer assignment
- POST /api/admin/claims/{claimId}/assign-officer — Assigns a claims officer; claim transitions to UNDER_REVIEW

4.9 Real-Time Notifications (FR-NOTIF)

The system shall deliver real-time push notifications to authenticated users via the Server-Sent Events (SSE) protocol. Notifications are triggered automatically by key lifecycle events such as policy status changes, claim decisions, payment confirmations, and concern raises. The JWT is passed as a URL query parameter due to the browser EventSource API's inability to send custom headers.

- GET /api/notifications/subscribe?token=<jwt> — Establishes the SSE stream connection
- GET /api/notifications — Returns all unread notifications for the authenticated user
- GET /api/notifications/all — Returns all notifications regardless of read status
- GET /api/notifications/count — Returns the current unread notification count
- PUT /api/notifications/mark-read — Marks selected or all notifications as read

4.10 PDF Generation (FR-PDF)

The system shall generate downloadable PDF documents for key artifacts. PDFs are produced using the iText 5.5.13.3 library and returned as application/pdf HTTP responses with appropriate Content-Disposition headers.

- GET /api/pdf/policy/certificate — Downloads the policy certificate PDF (policyId as query parameter)
- GET /api/payments/{id}/receipt — Downloads the payment receipt PDF for a specific payment

4.11 Audit Logging (FR-AUDIT)

The system shall maintain a comprehensive audit trail recording all significant operations performed by users across all roles, supporting compliance and administrative review.

- GET /api/audit — Returns all audit log entries (Admin access only)
- GET /api/audit/role/{role} — Returns audit entries filtered by the performing user's role
- GET /api/audit/my — Returns the authenticated user's own audit trail

5. Non-Functional Requirements

5.1 Security (NFR-SEC)

- NFR-SEC-01: All passwords shall be stored using the BCrypt adaptive hashing algorithm. Plain-text passwords shall never be stored or logged.
- NFR-SEC-02: All API endpoints except explicitly public routes shall require a valid, non-expired JWT in the Authorization: Bearer header.
- NFR-SEC-03: JWT access tokens shall have a configurable expiration time (JWT_EXPIRATION environment variable). Refresh tokens shall enable seamless re-authentication without re-login.

- NFR-SEC-04: CORS configuration shall restrict cross-origin requests to the known frontend origin only.
- NFR-SEC-05: All sensitive configuration values (JWT secret, SMTP credentials, Razorpay keys, database credentials) shall be externalized to environment variables and must never be committed to version control.
- NFR-SEC-06: File uploads shall be validated for permitted MIME types. Uploaded files shall be stored outside any web-accessible directory to prevent direct URL access.
- NFR-SEC-07: Razorpay payment callbacks shall be verified using HMAC-SHA256 signature validation before any policy activation is processed.

5.2 Performance (NFR-PERF)

- NFR-PERF-01: Standard CRUD API operations shall return responses within 2 seconds under a normal load of up to 200 concurrent users.
- NFR-PERF-02: Premium calculation endpoints shall respond within 1 second for up to 10 policy members.
- NFR-PERF-03: PDF generation endpoints (policy certificate, payment receipt) shall respond within 5 seconds.
- NFR-PERF-04: Real-time SSE notification delivery latency shall not exceed 3 seconds from the triggering server-side event to client receipt.

5.3 Reliability (NFR-REL)

- NFR-REL-01: The production system shall target a minimum availability of 99.5% during business hours.
- NFR-REL-02: All policy and claim state transitions shall be executed within database transactions to ensure atomicity. Partial state updates that leave the system in an inconsistent state are not permitted.
- NFR-REL-03: Payment processing shall include full error handling and failure event logging to prevent duplicate charge scenarios.

5.4 Scalability (NFR-SCALE)

- NFR-SCALE-01: The production deployment shall support a minimum of 500 concurrent active users.
- NFR-SCALE-02: The database schema shall use indexed foreign key columns on policyId, claimId, and userId for efficient query performance as data volume grows.

5.5 Usability (NFR-USE)

- NFR-USE-01: The Angular frontend shall render correctly on modern desktop browsers including Chrome 100+, Firefox 100+, and Edge 100+.
- NFR-USE-02: All user-facing forms shall provide real-time inline validation feedback without requiring a page refresh.
- NFR-USE-03: Each role-based dashboard shall display only the data, actions, and navigation items relevant to the authenticated user's assigned role.

5.6 Maintainability (NFR-MAIN)

- NFR-MAIN-01: The backend shall strictly follow the layered architecture pattern: HTTP request enters at the Controller layer and flows through Service to Repository, with no layer skipping permitted.
- NFR-MAIN-02: All REST API contracts shall be auto-documented via SpringDoc OpenAPI, accessible at /swagger-ui.html in any running instance.
- NFR-MAIN-03: Environment-specific configuration shall be managed via Spring Boot profiles: application.yaml for development, application-prod.yaml for production.

5.7 Portability (NFR-PORT)

- NFR-PORT-01: The application shall support switching between H2 in-memory (development) and MySQL 8.x (production) databases via configuration profile change alone, without requiring code modifications.
- NFR-PORT-02: The claim document upload directory path shall be fully configurable via the FILE_UPLOAD_DIR environment variable to support any deployment environment.

6. Complete API Specification

Base URL (Development): http://localhost:8080/api. All endpoints marked "Auth" require Authorization: Bearer <access_token> in the request header. Endpoints marked "Public" are accessible without authentication.

6.1 Authentication

Method	Endpoint	Access	Description
POST	/auth/register	Public	Register a new customer account
POST	/auth/login	Public	Authenticate and receive JWT tokens
POST	/auth/refresh	Public	Refresh an expired access token
POST	/auth/forgot-password	Public	Request an OTP email for password reset
POST	/auth/reset-password	Public	Reset password using a valid OTP

6.2 Insurance Plans

Method	Endpoint	Access	Description
GET	/plans	Public	List all active insurance plans
GET	/plans/{id}	Public	Get details of a specific plan
GET	/plans/type/{type}	Public	Filter plans by insurance type
POST	/admin/plans	Admin	Create a new insurance plan
PUT	/admin/plans/{id}	Admin	Update an existing plan
PATCH	/admin/plans/{id}/activate	Admin	Activate a plan
PATCH	/admin/plans/{id}/deactivate	Admin	Deactivate a plan

6.3 Premium Calculation

Method	Endpoint	Access	Description
POST	/premium/calculate	Public	Guest premium quote calculation
POST	/premium/calculate/me	Customer	Stored premium quote for authenticated customer
GET	/premium/my-quotes	Customer	Retrieve customer's saved quote history
GET	/underwriter/policy/{id}/calculate-quote	Underwriter	Base quote calculation for assigned policy
GET	/underwriter/calculate-quote/{id}	Underwriter	Detailed breakdown quote with all factors

6.4 Policies

Method	Endpoint	Access	Description
POST	/policies	Customer	Apply for a new policy (JSON)
POST	/policies/with-document	Customer	Apply with health check document (multipart)
GET	/policies/my-policies	Customer	Customer's own policies
GET	/policies	Admin/UW	All policies system-wide
GET	/policies/{id}	All Auth	Specific policy details
GET	/policies/{id}/members	All Auth	Members covered under a policy

PATCH	/policies/{id}/cancel	Customer	Cancel an active policy
POST	/policies/{id}/renew	Customer	Renew an expiring policy
PUT	/policies/{id}/reapply	Customer	Reapply after concern raised (multipart)
GET	/policies/{id}/document/download	All Auth	Download uploaded health check report

6.5 Admin Operations

Method	Endpoint	Access	Description
GET	/admin/dashboard	Admin	Admin dashboard metrics
POST	/admin/create-underwriter	Admin	Create underwriter account
POST	/admin/create-claims-officer	Admin	Create claims officer account
GET	/admin/customers	Admin	List all customer accounts
GET	/admin/underwriters	Admin	List all underwriter accounts
GET	/admin/claims-officers	Admin	List all claims officer accounts
PATCH	/admin/users/{id}/activate	Admin	Activate a user account
PATCH	/admin/users/{id}/deactivate	Admin	Deactivate a user account
GET	/admin/pending-applications	Admin	Policies awaiting UW assignment
POST	/admin/policies/{id}/assign-underwriter	Admin	Assign underwriter to a policy
GET	/admin/submitted-claims	Admin	Claims awaiting officer assignment
POST	/admin/claims/{id}/assign-officer	Admin	Assign claims officer to a claim

6.6 Underwriter Operations

Method	Endpoint	Access	Description
GET	/underwriter/dashboard	UW	Underwriter dashboard data
GET	/underwriter/profile	UW	View own profile
GET	/underwriter/policies	UW	All assigned policies
GET	/underwriter/pending-assignments	UW	Policies pending review
GET	/underwriter/plans	UW	View available insurance plans
POST	/underwriter/policy/{id}/send-quote	UW	Submit premium quote to customer

POST	/underwriter/policy/{id}/raise-concern	UW	Raise concern on an application

6.7 Claims

Method	Endpoint	Access	Description
POST	/claims	Customer	File a new claim (multipart)
GET	/claims/my-claims	Customer	Customer's own claims
GET	/claims/{id}	All Auth	View specific claim details
GET	/claims	Officer	All claims (officer view)
GET	/claims/pending	Admin/Officer	Pending unassigned claims
GET	/claims/status/{status}	All Auth	Claims filtered by status
PATCH	/claims/{id}/status	Admin	Manually update claim status
GET	/claims/{id}/documents	All Auth	List claim documents
GET	/claims/{cId}/documents/{dId}/download	All Auth	Download document by ID
POST	/claims/{id}/settle	Admin/Officer	Initiate claim settlement

6.8 Claims Officer Operations

Method	Endpoint	Access	Description
GET	/claims-officer/dashboard	Officer	Claims officer dashboard
GET	/claims-officer/queue	Officer	Queue of submitted claims
GET	/claims-officer/my-claims	Officer	Officer's assigned claims
GET	/claims-officer/my-claims/status/{s}	Officer	Filter assigned claims by status
GET	/claims-officer/my-decisions	Officer	Historical decision log
POST	/claims-officer/claim/{id}/pickup	Officer	Self-assign a claim
GET	/claims-officer/claim/{id}	Officer	View specific claim details
POST	/claims-officer/claim/{id}/review	Officer	Submit review decision

6.9 Payments and Razorpay

Method	Endpoint	Access	Description
POST	/payments	Customer	Standard premium payment submission
GET	/payments/my-payments	Customer	Customer's payment history
GET	/payments/{id}	All Auth	View specific payment details
GET	/payments	Admin	All payments (admin view)
GET	/payments/policy/{id}	All Auth	Payments for a specific policy
GET	/payments/{id}/receipt	All Auth	Download PDF payment receipt
POST	/razorpay/create-order	Customer	Create a Razorpay payment order
POST	/razorpay/verify	Customer	Verify Razorpay HMAC signature
POST	/razorpay/failure	Customer	Log a Razorpay payment failure

6.10 Notifications, Audit, and PDF

Method	Endpoint	Access	Description
GET	/notifications/subscribe?token=	All Auth	Subscribe to SSE notification stream
GET	/notifications	All Auth	Get unread notifications
GET	/notifications/all	All Auth	Get all notifications
GET	/notifications/count	All Auth	Get unread notification count
PUT	/notifications/mark-read	All Auth	Mark notifications as read
GET	/audit	Admin	All audit log entries
GET	/audit/role/{role}	Admin	Audit logs filtered by role
GET	/audit/my	All Auth	Own audit trail
GET	/pdf/policy/certificate	All Auth	Download policy certificate PDF

7. Environment and Configuration

7.1 Development Environment

Setting	Value
Database	H2 In-Memory — jdbc:h2:mem:insurewelldb
DDL Auto	create-drop (schema dropped and recreated on every application restart)
API URL	http://localhost:8080/api
Frontend URL	http://localhost:4200
Swagger UI	http://localhost:8080/swagger-ui.html
H2 Console	http://localhost:8080/h2-console
Upload Dir	uploads/claims (relative to application working directory)
Config File	Backend- Insure Well /src/main/resources/application.yaml

7.2 Production Environment Variables

Production is activated via the Spring prod profile (--spring.profiles.active=prod). The following environment variables must be configured in the deployment environment:

Environment Variable	Required	Purpose
DATABASE_URL	Yes	MySQL JDBC connection string
DATABASE_USERNAME	Yes	MySQL database username
DATABASE_PASSWORD	Yes	MySQL database password
JWT_SECRET	Yes	HMAC-SHA256 signing secret (minimum 256 bits)
JWT_EXPIRATION	Yes	Access token expiry in milliseconds
PORT	No	Server port (defaults to 8080)
FILE_UPLOAD_DIR	Yes	Absolute filesystem path for claim document storage
RAZORPAY_KEY_ID	Yes	Razorpay API key identifier
RAZORPAY_KEY_SECRET	Yes	Razorpay API secret for HMAC signature verification
SMTP_HOST	Yes	Mail server hostname
SMTP_PORT	Yes	Mail server port
SMTP_USERNAME	Yes	Mail server authentication username
SMTP_PASSWORD	Yes	Mail server authentication password

7.3 Security Notice

IMPORTANT: The development application.yaml file must not contain any real or live credentials. All sensitive values including JWT secrets, SMTP passwords, Razorpay API keys, and database credentials must be moved to environment variables before the repository is shared, deployed, or submitted. Any credentials that have already been committed to version control must be rotated and invalidated immediately.

7.4 Local Setup Guide

Backend Setup

1. Install Java Development Kit (JDK) version 21
2. Open a terminal and navigate to the Backend-Health_Insurance/ directory
3. Run the application using: .\mvnw spring-boot:run (Windows) or ./mvnw spring-boot:run (macOS/Linux)
4. The REST API will be available at http://localhost:8080/api
5. Swagger UI will be accessible at http://localhost:8080/swagger-ui.html

Frontend Setup

1. Install Node.js version 18 or higher and npm
2. Open a terminal and navigate to the Frontend-HealthInsurance/ directory
3. Install dependencies by running: npm install
4. Start the development server by running: npm start
5. The Angular application will be available at http://localhost:4200

8. Data Storage and File Management

8.1 Database Entity Model

Entity	Key Attributes	Relationships
User (base)	id, email, password, role, isActive, createdAt	Base class for Customer, Admin, Underwriter, ClaimsOfficer
Customer	demographics, address, phone, dateOfBirth	Has many Policies; has many Claims
Admin	employeeId	No domain-specific child entities
Underwriter	employeeId, department	Has many assigned Policies
ClaimsOfficer	employeeId, department	Has many assigned Claims
InsurancePlan	name, type, coverageAmount, basePremium, isActive	Referenced by many Policies
Policy	policyNumber, status, startDate, endDate, premiumAmount	Belongs to Customer and Plan; has Payments, Members

PolicyMember	name, age, relationship, preExistingConditions	Belongs to Policy
Claim	claimNumber, amount, status, description, filedDate	Belongs to Policy and Customer; has many ClaimDocuments
ClaimDocument	fileName, filePath, fileType, uploadedAt	Belongs to Claim
Payment	amount, status, transactionId, paymentDate, method	Belongs to Policy
Notification	message, isRead, type, createdAt	Belongs to User
AuditLog	action, performedBy, role, timestamp, details	Belongs to User
PremiumQuote	amount, calculatedAt, breakdown	Belongs to Customer and Plan

8.2 File Storage Strategy

Claim documents and health check reports are stored on the server filesystem rather than the database to avoid performance degradation from storing binary data in relational tables. Only document metadata is stored in the database.

- Default development upload path: uploads/claims/<claimId>/<filename>
- Production upload path: configured via FILE_UPLOAD_DIR environment variable
- ClaimDocument database entity stores: fileName, filePath, fileType, and uploadedAt timestamp
- Policy health check reports are accessible via GET /api/policies/{id}/document/download
- Files are stored outside the web-accessible directory to prevent direct URL access

9. Constraints and Assumptions

9.1 Constraints

- The Angular 21 frontend requires Node.js version 18 or higher for local development and build tooling.
- The Spring Boot backend requires Java Development Kit (JDK) version 21 and Apache Maven for compilation and execution.
- Razorpay payment gateway integration requires an active, verified Razorpay merchant account for live transaction processing.
- File uploads are subject to Spring Boot's default multipart request size limit of 10 MB per file, configurable in application.yaml.
- JWT signing secrets must be a minimum of 256 bits in length to satisfy the HMAC-SHA256 algorithm requirements.
- The SSE notification subscription endpoint accepts the JWT as a URL query parameter because the browser EventSource API does not support custom request headers.

9.2 Assumptions

- All users access the system through a modern, standards-compliant desktop web browser such as Google Chrome 100+, Mozilla Firefox 100+, or Microsoft Edge 100+.
- For production deployments, a MySQL version 8.x database server is provisioned, accessible, and properly configured before deployment.
- A working SMTP mail server with valid credentials is available for delivering OTP codes and system notification emails.
- Valid Razorpay API credentials (Key ID and Key Secret) are obtained and configured before enabling live payment processing.
- The system serves a single corporate organization and manages multiple customers within that organization. Multi-tenancy (multiple independent organizations) is not a requirement for version 1.0.
- All health documents uploaded by customers are treated as medically sensitive. Access to these documents is restricted by the RBAC system and available only to authorized roles.

10. Document Revision History

Version	Date	Author	Description
1.0	March 2026	Capstone Team	Initial release — complete SRS based on Insure Well implementation

— End of Document —